Design and Analysis of Algorithms

Dynamic Programming

National University of Computer and Emerging Sciences, Islamabad

Divide and Conquer

- It deals (involves) three steps at each level of recursion:

  Divide the problem into a number of subproblems

  Conquer the subproblems by solving them recursively

  Combine the solution to the subproblems into the solution for original subproblems.

  MergeSort, Quick Sort, Binary Search

Recursive Definition of the Fibonacci Numbers

The Fibonacci numbers are a series of numbers as follows:

fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
...

$$fib(n) = \begin{cases} 1, & n <= 2 \\ fib(n-1) + fib(n-2), & n > 2 \end{cases}$$

fib(3) = 1 + 1 = 2
fib(4) = 2 + 1 = 3
fib(5) = 2 + 3 = 5

Fibonacci Numbers

EXPONENTIAL — BAD!

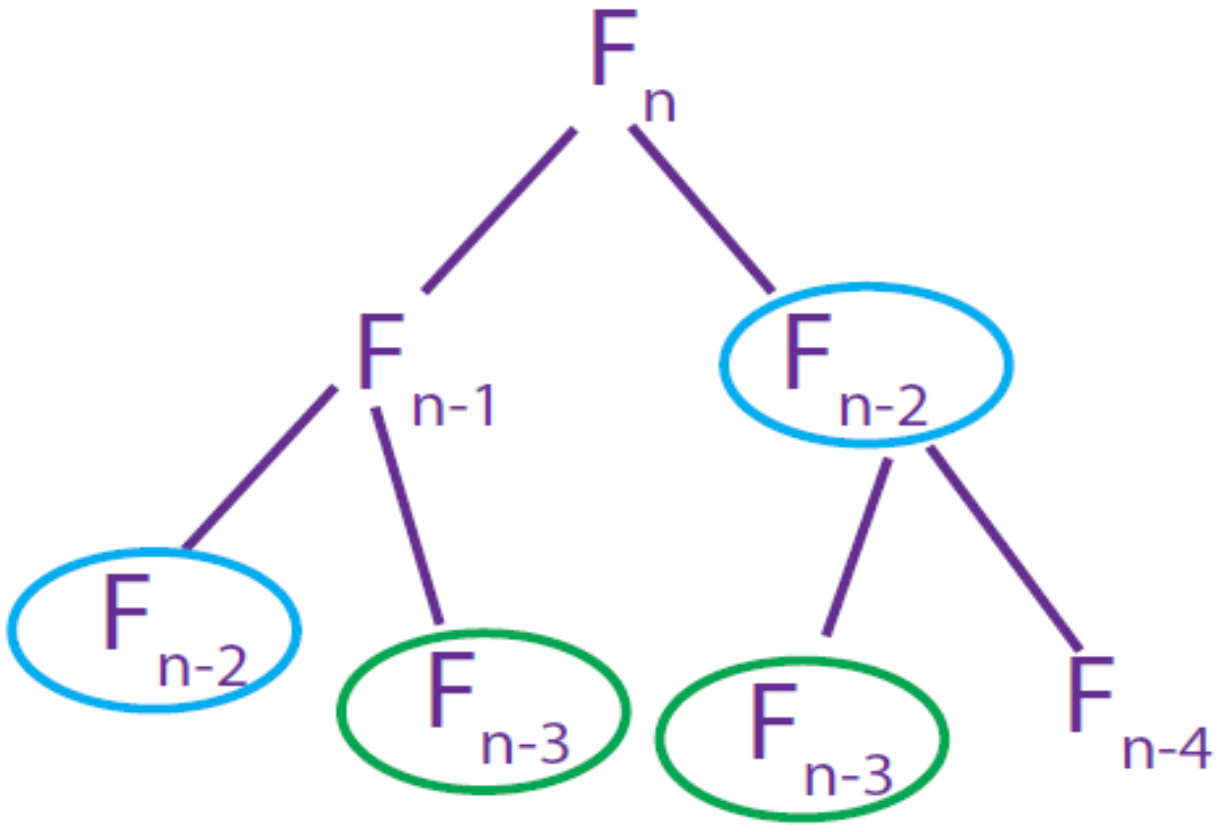


Figure 1: Naive Fibonacci Algorithm.

Recursive Definition of the Fibonacci Numbers

$$T(n) = T(n-1) + T(n-2) + O(1)$$

$\underline{\text{fib}}(n)$:

if $n \leq 2$: return $f = 1$

else: return $f = \text{fib}(n-1) + \text{fib}(n-2)$

$$\Longrightarrow T(n) = T(n-1) + T(n-2) + O(1) \geq F_n \approx \varphi^n$$

$$\geq 2T(n-2) + O(1) \geq 2^{n/2}$$

EXPONENTIAL — BAD!

Dynamic Programming

- **Dynamic programming like the divide and conquer method, solves problem by combining the solutions of sub problems**

- **Divide and conquer method**
  - **partition the problem into independent sub problems**
  - **solves the sub problems recursively**
  - **combine their solutions to solve the original problem**

Dynamic Programming

- **Dynamic programming is applicable,**
  - **when the sub-problems are NOT independent, that is when sub-problems share sub sub-problems.**
- **It is making a set of choices to arrive at optimal solution.**
- **If sub-problems are not independent,**
  - **we have to further divide the problem.**
- **In worst case,**
  - **we may end-up with an exponential time algorithm.**

Dynamic Programming

It involves the sequence of four steps:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution bottom-up.
4. Construct an optimal solution from computed information

Divide & Conquer

It deals (involves) three steps at each level of recursion:

Divide the problem into a number of subproblems.

Conquer the subproblems by solving them recursively.

Combine the solution to the subproblems into the solution for original subproblems.

Dynamic Programming

- It is non Recursive.
- It solves subproblems only once and then stores in the table.
- It is a Bottom-up approach.
- In this subproblems are interdependent.
- **For example:** Matrix Multiplication.

Divide & Conquer

- It is Recursive.
- It does more work on subproblems and hence has more time consumption.
- It is a top-down approach.
- In this subproblems are independent of each other.
- **For example:** Merge Sort & Binary Search etc

Dynamic Programming

- **Frequently, there is a polynomial number of sub-problems, but they get repeated.**
- **A dynamic programming algorithm solves every sub-problem just once and then saves its answer in a table, thereby avoiding the work of re-computing the answer every time the sub-problem is encountered**
- **So we end up having a polynomial time algorithm.**
- **Which is better, Dynamic Programming or Divide & conquer?**

Optimization Problems

- **Dynamic problem is typically applied to *Optimization Problems***

- **In optimization problems there can be many possible solutions. Each solution has a value and the task is to find the solution with the optimal (Maximum or Minimum) value. There can be several such solutions.**

4 steps of Dynamic Programming Algorithm

1. Characterize the structure of an optimal solution.

2. Recursively define the value of an optimal solution.

3. Compute the value of an optimal solution bottom-up.

4. Construct an optimal solution from computed information

Often only the value of the optimal solution is required so step-4 is not necessary.

If we need only the value of an optimal solution, and not the solution itself, then we can omit step 4.

Recursive Algorithm (seen earlier)

- **Takes Exponential time, seen few lectures back!**
- **Actual sub problems are polynomial (O(n)) but they get repeated**
- **Sub problems are not INDEPENDENT.**
- **Sub problems share sub-sub problems.**
- **We can solve it using Dynamic programming.**

Memoized DP Algorithm

- Remember, remember
  - memo = { }
  - fib(n): if n in memo: return memo[n]
  - else: if $n \leq 2$: f = 1
  - else: f = fib(n − 1) + fib(n − 2)
  - memo[n] = f
  - return f

Benefit of Dynamic Programming

```
// to compute fibonacci(n)
int[] d = new int[n+1];
for (int j = 2; j<=n; j++)
                        d[j] = -1;
d[0] = 1;
d[1] = 1;

int fibonacci(int n)
{
    if (d[n-1] < 0)
                        d[n-1] = fibonacci(n-1);

    d[n] = d[n-1] + d[n-2];
    return (d[n]);
}
```

Memoized DP Algorithm

- $\Longrightarrow$ fib(k) only recurses <u>first</u> time called, $\forall k$
- $\Longrightarrow$ only n nonmemoized calls: $k = n, n-1, \ldots, 1$
- memoized calls free ($\Theta(1)$ time)
- $\Longrightarrow$ $\Theta(1)$ time per call (ignoring recursion)

POLYNOMIAL — GOOD!

Memoized DP Algorithm

* DP $\approx$ recursion + memoization
  - memoize (remember) & re-use solutions to subproblems that help solve problem
    - in Fibonacci, subproblems are $F_1, F_2, \ldots, F_n$
* $\Longrightarrow$ time = # of subproblems $\cdot$ time/subproblem
  - Fibonacci: # of subproblems is n, and time/subproblem is $\Theta(1) = \Theta(n)$ (ignore recursion!).

Memoization

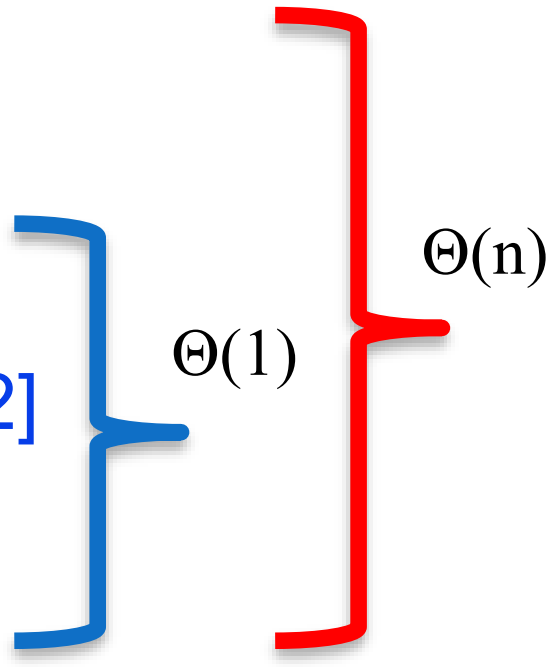
- **A variation of Dynamic Programming and is TOP-DOWN**
- **Memoize the natural, but inefficient, recursive algorithm.**
- **Maintains an entry in the table for the solution to each sub-problem.**
- **Offers efficiency while maintaining top-down strategy.**

Memoization

- **A special character stored at each empty solution space.**
- **When the sub-problem is first encountered, it's solution is computed and stored.**
- **Each subsequent time the sub-problem is encountered, its solution is *looked up*.**
- **Time to search the sub-problem solutions?**

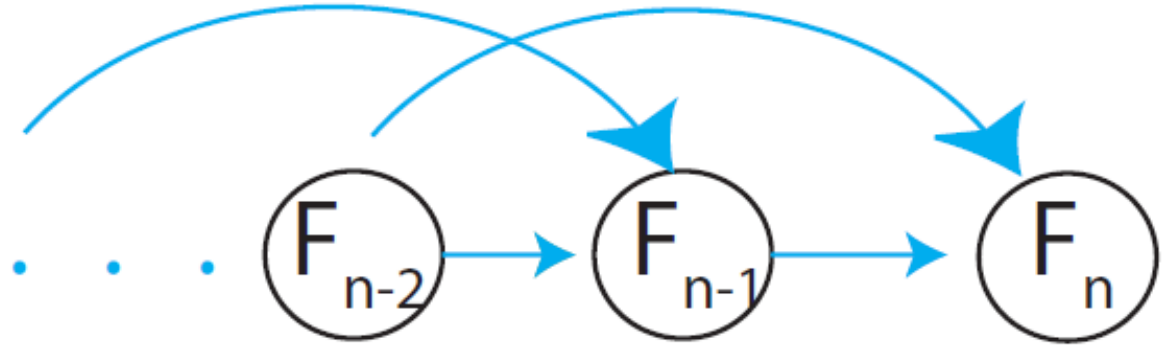
Bottom-up DP Algorithm

- fib = { }
- for k in [1, 2, . . . , n]:
  - if k ≤ 2: f = 1
  - else: f = fib[k − 1] + fib[k − 2]
  - fib[k] = f
- return fib[n]



Bottom-up DP Algorithm

- exactly the same computation as memoized DP (recursion “unrolled”)



- practically faster: no recursion
- analysis more obvious
- can save space: just remember last 2 fibs $\Rightarrow \Theta(1)$

Benefit of Dynamic Programming

```
// to compute fibonacci(n)
int[] d = new int[n+1];
for (int j = 2; j<=n; j++)
                    d[j] = -1;
d[0] = 1;
d[1] = 1;

int fibonacci(int n)
{
    if (d[n-1] < 0)
                    d[n-1] = fibonacci(n-1);

    d[n] = d[n-1] + d[n-2];
    return (d[n];
}
```

```
// to compute fibonacci(n)
int[] d = new int[n+1];
for (int j = 2; j<=n; j++)
            d[j] = -1;
d[0] = 1;
d[1] = 1;

int fibonacci(int n)
{
    for(int j = 2; j<= n; j++)
            d[j] = d[j-1]+d[j-2];

    return (d[n]);
}
```

Dynamic-programming hallmark #1

Optimal substructure

An optimal solution to a problem (instance) contains optimal solutions to subproblems.

Dynamic-programming hallmark #2

Overlapping subproblems

A recursive solution contains a "small" number of distinct subproblems repeated many times.

- **Rod Cutting Problem using Dynamic Programming**

Example: **rod cutting**

We are given prices p_i and rods of length i:

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Question: We are given a rod of length n, and want to maximize revenue, by cutting up the rod into pieces and selling each of the pieces.

Example: we are given a 4 inches rod. Best solution to cut up? We'll first list the solutions:

We'll first list the solutions:

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

1.) Cut into 2 pieces of length 2: $p_2 + p_2 = 5 + 5 = 10$

2.) Cut into 4 pieces of length 1: $p_1 + p_1 + p_1 + p_1 = 1 + 1 + 1 + 1 = 4$

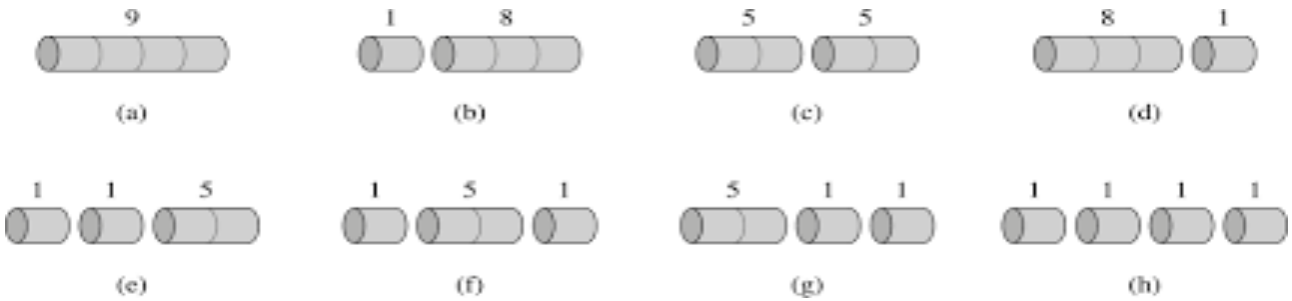
3-4.) Cut into 2 pieces of length 1 and 3 (3 and 1): $p_1 + p_3 = 1 + 8 = 9$ $\quad p_3 + p_1 = 8 + 1 = 9$

5.) Keep length 4: $p_4 = 9$

6-8.) Cut into 3 pieces, length 1, 1 and 2 (and all the different orders)

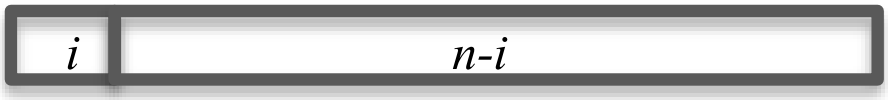
$p_1 + p_1 + p_2 = 7$ $\quad p_1 + p_2 + p_1 = 7$ $\quad p_2 + p_1 + p_1 = 7$

Total: 8 cases for $n = 4$ (= 2^{n-1}). We can slightly reduce by always requiring cuts in non-decreasing order. But still a lot!



Note: We've computed a brute force solution; all possibilities for this simple small example. But we want more optimal solution!

One solution: recurse on further



What are we doing?
- Cut rod into length i and n-i
- Only remainder n-i can be further cut (recursed)

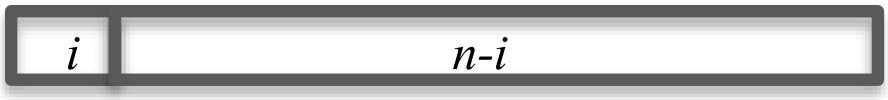
We need to define:

a.) **Maximum revenue** for log of size n: $\boldsymbol{r_n}$ (that is the solution we want to find).

b.) **Revenue (price)** for single log of length i: $\boldsymbol{p_i}$

Example: If we cut log into length *i* and *n-i*:

recurse on further



Revenue: $p_i + r_{n-i}$ Can be seen by recursing on *n-i*

What are we going to do?

There are many possible choices of *i*:

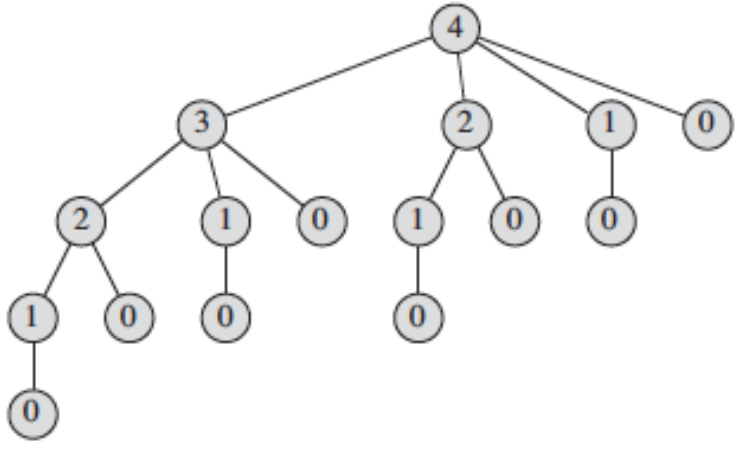
$$r_n = \max\begin{Bmatrix} p_1 + r_{n-1} \\ p_2 + r_{n-2} \\ \dots \\ p_n + r_0 \end{Bmatrix}$$

Recursive (top-down) pseudo code:

```
CUT-ROD(p, n)
1  if n == 0
2      return 0
3  q = -∞
4  for i = 1 to n
5      q = max(q, p[i] + CUT-ROD(p, n - i))
6  return q
```

Problem? Slow runtime (it's essential brute force)!

But why? Cut-rod calls itself repeatedly with the same parameter values (tree):



- Node label: size of the subproblem called on
- many subproblems are called repeatedly (subproblem overlap)
- Number of nodes exponential in n (2^n). therefore exponential number of calls.

Therefore … Dynamic Programing Approach:

- Recursive solution is inefficient, since it repeatedly calculates a solution of the same subproblem (overlapping subproblem).
- Instead, solve each subproblem only once AND save its solution. Next time we encounter the subproblem look it up in a hashtable or an array (Memoization, recursive top-down solution).
- We will also talk about a second solution where we save the solution of subproblems of increasing size (i.e. in order) in an array. Each time we will fall back on solutions that we obtained in previous steps and stored in an array (bottom-up solution).

Recursive top-down solution: **Cut-Rod with Memoization**

- **Step 1** Initialization:

```
MEMOIZED-CUT-ROD(p, n)
1  let r[0..n] be a new array
2  for i = 0 to n
3      r[i] = -∞
4  return MEMOIZED-CUT-ROD-AUX(p, n, r)
```

Creates array for holding memoized results, and initialized to minus infinity. Then calls the main auxiliary function.

Step 2: The main auxiliary function, which goes through the lengths, computes answers to subproblems and memoizes if subproblem not yet encountered:

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
1  if r[n] ≥ 0
2      return r[n]
3  if n == 0
4      q = 0
5  else q = −∞
6      for i = 1 to n
7          q = max(q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n − i, r))
8  r[n] = q
9  return q
```

Bottom-up solution: **Bottom Up Cut-Rod**

Each time we use previous values form arrays:

```
BOTTOM-UP-CUT-ROD(p, n)
1  let r[0..n] be a new array
2  r[0] = 0
3  for j = 1 to n
4      q = -∞
5      for i = 1 to j
6          q = max(q, p[i] + r[j - i])
7      r[j] = q
8  return r[n]
```

Check if value already known or memoized

Compute maximum revenue if it hasn't already been computed.

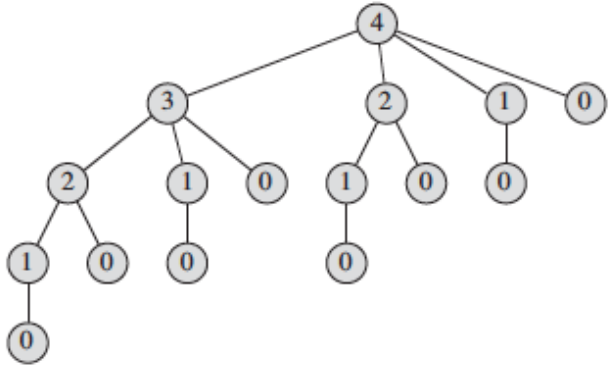
Saving value

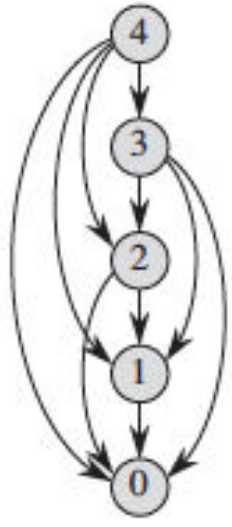
Run time: For bottom-up and top-down approach $O(n^2)$

Why (double nested loop)?

We can also view the subproblems encountered in graph form.

- We reduce the previous tree that included all the subproblems repeatedly





- each vertex represents subproblem of a given size
- Vertex label: subproblem size
- Edges from *x* to *y*: We need a solution for subproblem *x* when solving subproblem *y*

Run time: **Can be seen as number of edges $O(n^2)$**

Note: Run time is a combination of number of items in table *(n)* and work per item *(n)*.
The work per item because of the max operation (needed even if the table is filled
And we just take values from the table)
is proportional to *n* as in the number of edges in graph.

Reference

- **Introduction to Algorithms**
  - **15.1 (Rod Cutting)**
  - **Chapter # 15**
  - **Thomas H. Cormen**